Приложение за геометричен анализ на триизмерни (3D) полигони и проверка за тяхната конгруентност (еднаквост на формата и размера, независимо от позицията и ориентацията им в пространството).

1. Обработка на входните данни

Входните данни се обработват от функцията parse_polygon_text. Процесът включва следните стъпки:

- **Извличане на координати:** Приложението чете текстов низ, съдържащ координати. То е гъвкаво и може да разделя стойностите по интервали, запетаи или точки и запетаи, като игнорира празни редове и коментари.
- **Валидация:** Проверява дали всеки ред съдържа точно три координати (X, Y, Z) и дали има минимум три точки за съставяне на полигон.
- **Нормализация:** Данните се преобразуват в числови масиви (numpy.ndarray). Ако първата и последната точка съвпадат (т.е. контурът е изрично затворен във входните данни), дублиращата се последна точка автоматично се премахва.
- **Цветово кодиране:** Към всеки полигон се присвоява цвят въз основа на името на файла (color_for_filename), което улеснява визуалното разпознаване по-късно.

2. Последователност на действията

Приложението изпълнява действията в следния логически ред:

1. **Проекция и проверка за планарност (равнинност):**
    - Чрез функцията fit_plane се използва singular value decomposition (SVD), за да се намери най-близката равнина, към която принадлежат 3D точките.
    - Точките се проектират в 2D координатна система върху тази равнина. Изчислява се максималното отклонение на 3D точките от равнината, за да се потвърди дали полигонът е строго планарен.
2. **Геометричен анализ:**
    - **Дегенерация:** Изчислява се ориентираното лице (signed_area). Ако то е близко до нула, полигонът се маркира като дегенерирал (невалиден).
    - **Самопресичане:** Функцията has_self_intersections проверява дали страните на полигона се пресичат една с друга, използвайки тестове за ориентация на отсечки.
    - **Изпъкналост:** Ако не се самопресича, се проверява дали полигонът е изпъкнал (is_convex) чрез векторни произведения на съседни ръбове. (не се използва в проверката, но дава полезна информация за други анализи)
    - Всички тези данни се запазват в структурата PolygonRecord.

3. **Тест за конгруентност (test_congruence):**
    - *Предварителен филтър:* За бързо отхвърляне на очевидно различни полигони, се сравняват сортираните дължини на страните им.
    - *Прокрустов анализ (Procrustes Analysis):* За да се намери точно съвпадение, вторият полигон се тества във всички възможни циклични отмествания на върховете, както и в обратен ред (отчитайки възможна огледална симетрия). За всяка комбинация се изчислява оптималната ротация и транслация, която наслагва втория полигон върху първия.
    - Ако максималната грешка след най-доброто напасване е под зададения толеранс (atol), полигоните се обявяват за конгруентни.

3. Извеждане на резултата

Резултатите се извеждат по два начина: като структурирани данни и визуално.

- **Данни:** Функцията за конгруентност връща речник (dictionary), който съдържа булева стойност (congruent), обяснение (reason), изчислената грешка (max_err) и матрицата с трансформираните координати на втория полигон, които го напасват върху първия.
- **Визуализация:** Функцията plot_pair използва библиотеката matplotlib, за да генерира графика с два панела:
    - **2D изглед:** Показва проектираните в равнината полигони.
    - **3D изглед:** Показва оригиналните полигони в триизмерното пространство.
    - Ако полигоните са конгруентни, вторият полигон се изчертава допълнително с черна пунктирана линия в своя *трансформиран вид*, за да се покаже визуално как точно се припокрива с първия.

4. Разлика между алгоритмичния (Прокрустов) подход и аналитичната проверка

Прложението използва изчислителен (алгоритмичен) метод за проверка на конгруентността, който се отличава съществено от класическите аналитични методи (напр. признаците за еднаквост на триъгълници или аналитичното доказване чрез изразяване на координатите с формули). Основните разлики са:

1. **Използване на толеранс (Приблизителност срещу Точност):**

- **Аналитичен метод:** В класическата математика две отсечки или ъгли са или абсолютно равни, или не. Доказателствата боравят с точни стойности (напр. координатите са представени като точни дроби или радикали).
- **Това приложение (Алгоритмичен метод):** Тъй като работи с числа с плаваща запетая (floating-point numbers), които имат присъщи закръгляния в компютърната памет, приложението използва *толеранс за грешка* (в кода: atol и eps). То не търси перфектно

съвпадение до безкраен знак след десетичната запетая, а пита: *"Ако наложим двата полигона един върху друг, максималното разстояние между съответните им върхове по-малко ли е от допустимата грешка (atol)?"*. Това го прави практично за реални данни от сензори или CAD модели.

2. **Намиране на трансформацията (Глобална оптимизация срещу Локално сравнение):**

- **Аналитичен метод:** Обикновено сравняваме елементите един по един (напр. страна-ъгъл-страна). Ако всички съответни елементи са равни, заключаваме, че фигурите са еднакви.
- **Това приложение (Алгоритмичен метод):** Приложението използва **Прокрустов анализ** (Procrustes Analysis). Той взима целия масив от точки на втория полигон и чрез матрични операции (SVD) намира *оптималната* ротация и транслация, която минимизира разстоянието до първия полигон. Тоест, алгоритъмът се опитва "физически" да премести и завърти втората фигура така, че да пасне възможно най-добре върху първата, и след това измерва остатъчното разминаване.

3. **Проблем със съответствието на върховете:**

- **Аналитичен метод:** Обикновено предварително знаем (или дефинираме) кои върхове от първата фигура на кои върхове от втората съответстват (напр. $\triangle ABC \cong \triangle DEF$).
- **Това приложение (Алгоритмичен метод):** Компютърът не знае откъде "започва" вторият полигон. Затова трябва да се изпробват всички възможни отмествания (циклични ротации на масива с координати) и дори обратния ред на точките (заради възможна огледална симетрия), за да намери правилното съответствие между върховете на двете фигури. Това е т.нар. "търсене с пълно изчерпване" (brute-force search) по отношение на индексите на върховете, съчетано с оптимално напасване за всяка комбинация.

Накратко, докато аналитичната проверка разчита на логически стъпки и точни равенства на съставните елементи, приложението използва глобално "напасване" (best-fit alignment) и измерване на максималната грешка, за да се справи с неточностите на компютърната аритметика и липсата на предварителна информация за ориентацията.